Behind the Predictions

Speaker : **Ridwan Amure**

> **Behind the Scenes series, Session 2**
> Last session we learned that a model reads and writes **tokens**. This session is about what it does with them.

> ## The one idea to leave with
>
> **An LLM is a next-token predictor.** Given all the text so far, it estimates how likely every possible next token is, picks one, adds it to the text, and repeats. One token at a time, with no plan written down anywhere else.
>
> Almost everything an LLM does, good and bad, follows from that one loop: fluent writing, hallucinations, weak arithmetic, different answers to the same question, why prompts and examples matter, why "think step by step" helps, and how agents call tools.

Session map

#	Section	Question it answers
1	The one idea: predict the next token	What is the model actually doing?
2	What a prediction looks like	What comes out of the model at each step?
3	One step at a time	How do single predictions become a paragraph?
4	How the model learned to predict	Where do the probabilities come from?
5	Choosing the next token: temperature and sampling	Why do I get a different answer each time?
6	From autocomplete to assistant	If it's just predicting text, why does it answer my questions?
7	One idea, many behaviours	Hallucinations, maths errors, loops, tool calls, explained
8	What this means for builders	How do we design around it?
9	Myths and reality	"It's just autocomplete", "it looks things up", "it remembers me"
10	Hands-on demos	See the probabilities yourself
11	Takeaways, discussion and glossary	

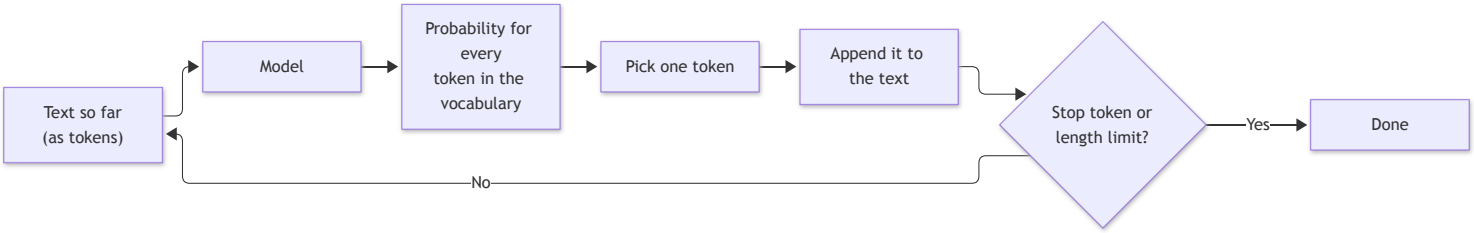
> **About the examples:** Every probability and generated text in this doc is real output from **GPT-2 small** (OpenAI, 2019: 124 million parameters, 50,257-token vocabulary), run locally with Hugging Face `transformers`. We use it because it is open, small enough to run on a laptop, and lets us see the raw probabilities. It is also a **base model** (no assistant training), so its mistakes are easy to see. Today's models are far larger and better trained, but they generate text with the **same loop**.

1. The one idea: predict the next token

You already use a next-word predictor every day: your phone keyboard. Type *"I'm on my"* and it suggests *"way"*. An LLM does the same job, with three big differences:

	Phone keyboard	LLM
What it looks at	The last word or two	The **entire** context: system prompt, conversation, documents, tool results (up to hundreds of thousands of tokens)
What it learned from	Your typing and a small language model	A large share of the public text on the internet, books and code (**trillions** of tokens; Meta's Llama 3 was trained on about 15 trillion)
Who picks the next word	You tap a suggestion	The model picks one itself, appends it, and keeps going

That last row is the key. Let the predictor **pick its own suggestion and keep going**, and autocomplete becomes a writer.



Hold on to this diagram. Everything else in this session is a detail of one of these boxes.

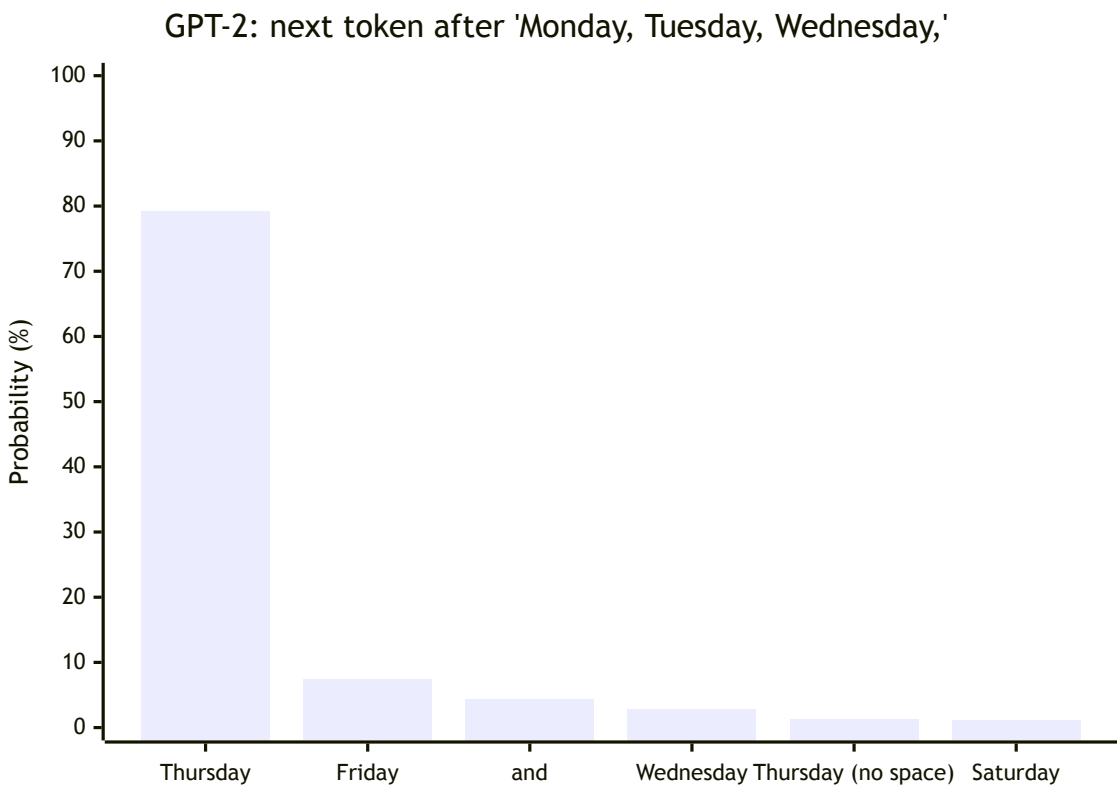
2. What a prediction looks like

At each step the model does **not** output a word. It outputs a **probability for every token in its vocabulary**: 50,257 numbers for GPT-2, around 100K–200K for modern models. They add up to 100%.

Example: an easy prediction

Prompt: `Monday, Tuesday, Wednesday,`

Next token	Probability
`Thursday`	**79.23%**
`Friday`	7.39%
`and`	4.37%
`Wednesday`	2.80%
`Thursday` (no space)	1.32%
`Saturday`	1.08%
...50,251 other tokens	the remaining ~3.8%



The model is very sure here, because this pattern appeared many times in its training text.

Example: an open-ended prediction

Prompt: `My favourite food is`

Next token	Probability
`the`	9.18%
`a`	3.04%
`chicken`	1.92%
`my`	1.43%
`always`	1.42%
`to`	1.16%

No token is above 10%. Many continuations are reasonable, so the probability is **spread out**. That spread shows how uncertain the model is.

Context changes everything

The same word, `bank`, in two different contexts:

Prompt	Top predictions
`I went to the bank to`	`get` 17.1% · `buy` 12.3% · `pay` 6.7% · `make` 5.3% · `check` 4.0% · `withdraw` 3.4% · `borrow` 2.8%

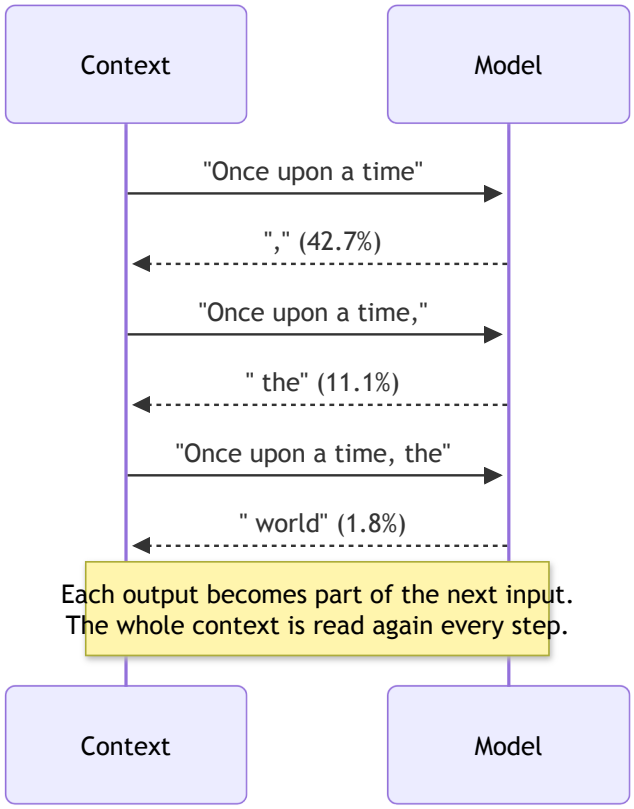
Prompt	Top predictions
`We sat on the river bank and`	`watched` 16.5% · `we` 6.0% · `looked` 4.7% · `listened` 3.5% · `saw` 3.1% · `talked` 2.9%

Nobody told the model there are two meanings of "bank". It learned that `river` changes what usually comes next. This is the core skill of an LLM: **using the whole context to shape the next prediction.**

3. One step at a time

Here is GPT-2 writing a sentence, one token per step. At each step we show the top 3 predictions and always pick the most likely (bold).

Step	Text so far	Top 3 predictions
1	`Once upon a time`	**`,` 42.7%** · `the` 6.5% · `I` 4.1%
2	`Once upon a time,`	**`the` 11.1%** · `I` 6.4% · `it` 4.0%
3	`Once upon a time, the`	**`world` 1.8%** · `people` 0.6% · `two` 0.5%
4	`Once upon a time, the world`	**`was` 22.1%** · `'s` 6.5% · `would` 6.0%
5	`Once upon a time, the world was`	**`a` 5.6%** · `filled` 4.1% · `full` 3.0%
6	`Once upon a time, the world was a`	**`place` 4.2%** · `different` 2.8% · `land` 2.6%
7	`Once upon a time, the world was a place`	**`of` 61.4%** · `where` 17.2% · `that` 2.4%
8	`Once upon a time, the world was a place of`	**`great` 2.8%** · `peace` 2.6% · `chaos` 2.1%



Three things to notice:

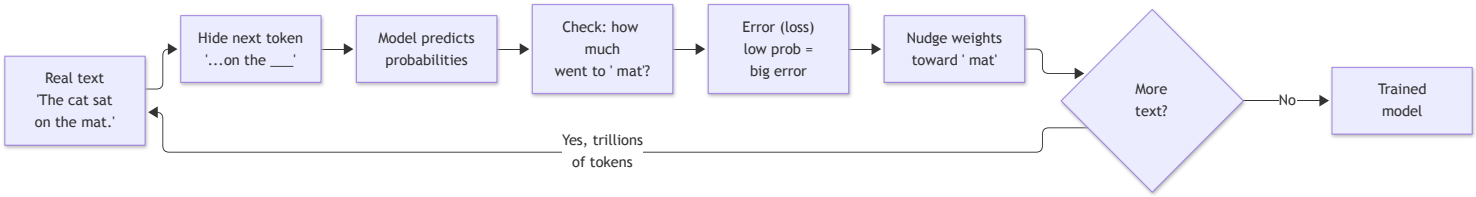
1. **Each chosen token becomes input for the next step.** The model's own output steers what it says next.
2. **Some steps are near-certain** (step 7: `of` at 61%) and **some are wide open** (step 3: `world` at only 1.8%). The model is confident in some places and guessing in others, within one sentence.
3. **There is no plan or draft.** The model didn't decide on "great beauty" at step 1. It committed to one token at a time, and **it cannot go back and edit** a token once it has been chosen.

Why this shapes what you see

- **Streaming:** Chat apps show text appearing word by word because it really is produced one token at a time.
- **Output is slow and costly:** As covered in Session 1, every output token needs another pass through the model. That's why output tokens cost about 5× more than input.
- **Early tokens matter most:** A bad early token pushes all later predictions in a bad direction. The model then continues it fluently.

4. How the model learned to predict

Training is a guessing game played trillions of times



There is no labelled dataset of right answers. **The text is its own answer key**: every next token in every document is a free training example. That's why models can learn from so much data.

What the model "thinks" of a real sentence

How much probability GPT-2 gives each actual next token in two sentences:

Position	`The cat sat on the mat.`	`The cat sat on the banana.`
`cat` after `The`	0.01%	0.01%
`sat`	0.07%	0.07%
`on`	22.40%	22.40%
`the`	45.37%	45.37%
`mat` **vs** `banana`	**0.40%**	**0.004%**
`.`	6.31%	3.92%

`mat` is about **100× more likely** than `banana`. Nobody wrote a rule saying cats sit on mats. The model picked it up from seeing many sentences, and training is what produces these numbers.

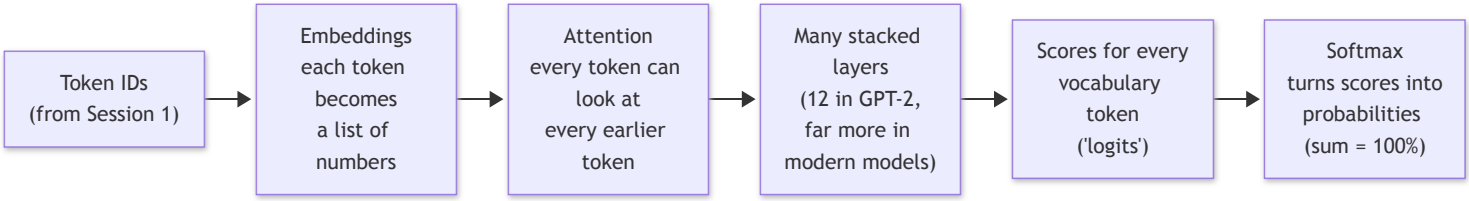
Why prediction leads to more than grammar

To predict the next token **well** across all of that text, a model has to pick up a lot as a side effect:

To predict this well...	...the model has to learn
`The capital of France is` → `Paris`	Facts about the world
`def add(a, b): return` → `a + b`	Programming patterns
`She dropped the glass and it` → `shattered`	Cause and effect, everyday physics
`Ẹ káàárọ̀, ṣé` → ...	Other languages
The last line of a proof	Some reasoning patterns

That's how "just predicting the next token" ends up with surprisingly broad abilities. It is also why the abilities are **uneven**: the model is strong where its training text was rich and weak where it was thin.

Inside one prediction (a light view)



- **Attention** is how `river` reaches forward to change what `bank` means. Each new prediction can draw on any earlier token in the context window.
- **The model only sees earlier tokens**, never later ones. It reads left to right because it was trained to predict what comes *next*.
- The model's "knowledge" is stored in its **weights** (numbers fixed after training), not in a database it searches.

5. Choosing the next token: temperature and sampling

The model produces probabilities. **Something else decides which token to actually use.** This step is called **decoding** or **sampling**, and it's controlled by settings you can often adjust.

Strategy 1: Greedy (always pick the top token)

Real GPT-2 output, greedy, from `Once upon a time` :

> Once upon a time, the world was a place of great beauty and great danger. The world was a place of great danger, and the world was a place of great...

Greedy is predictable, but it often gets **stuck in loops**. The same happens with `I like to` :

> I like to think that I'm a good person. I'm not a bad person. I'm not a bad person. I'm not a bad person. I'm not a bad person...

Once a phrase appears, repeating it becomes the most likely continuation, and the loop feeds itself.

Strategy 2: Sampling (roll weighted dice)

Pick a token **at random, in proportion to its probability**. A token at 20% gets picked about 1 time in 5. Three real runs of GPT-2 from the same prompt:

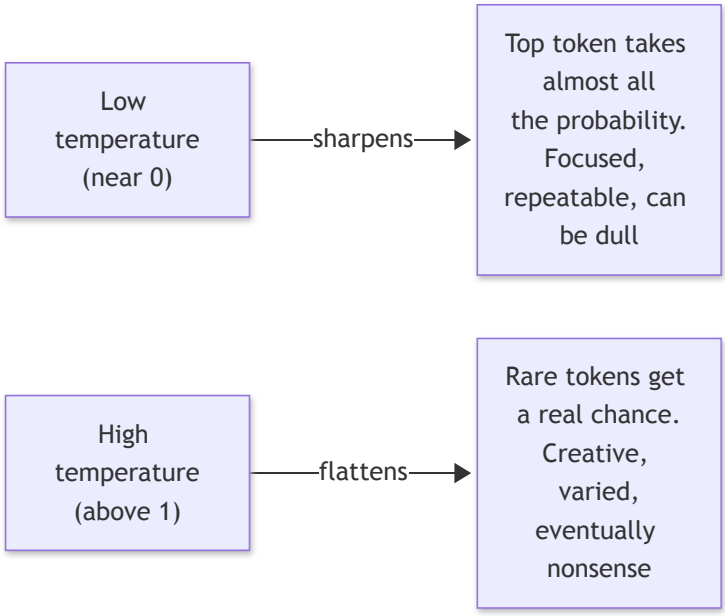
Run	`Once upon a time …`
1	`, my role changed. I took the role of an assimilator while learning about the scavenger armor`
2	`people believed there were four angry or holier than thou People who would issue instructions for Heaven only,`
3	`there was none left of Babylon, the third great capital, and the beginning of the process of conquest`

Same model, same prompt, three different stories. This is why ChatGPT or Claude gives you a different answer when you regenerate.

Temperature: how adventurous the dice are

Temperature reshapes the probabilities before sampling. Real GPT-2 numbers for `My favourite food is` :

Next token	T = 0.2 (cautious)	T = 0.7	T = 1.0 (raw)	T = 1.5 (wild)
`the`	**99.53%**	34.43%	9.18%	1.49%
`a`	0.40%	7.11%	3.04%	0.71%
`chicken`	0.04%	3.67%	1.92%	0.52%
`my`	0.01%	2.42%	1.43%	0.43%
`always`	0.01%	2.38%	1.42%	0.43%



What too much temperature looks like (GPT-2, T = 2.0):

> Once upon a time **upset playground shape griippy principal UNDER savvy verbal proofs verbally assimil c AUTpose taunt**

Other common controls

Setting	What it does
Top-k	Only consider the k most likely tokens (e.g. top 40), then sample
Top-p (nucleus)	Only consider the smallest set of tokens whose probabilities add up to p (e.g. 90%), then sample
Max tokens	Stop after this many output tokens
Stop sequences	Stop when a specific string is produced
Structured output / constrained decoding	Block any token that would break a required format (e.g. invalid JSON). The model can only pick from tokens that keep the output valid

> **Note for Claude builders:** Recent Claude models (Opus 4.7 and later, Sonnet 5) no longer accept `temperature`, `top_p` or `top_k`. The provider manages sampling for you, and you steer behaviour through the prompt and the `effort` setting instead. Older models such as Claude Haiku 4.5 still accept them. Other providers vary, so check the docs for the model you use.

6. From autocomplete to assistant

A raw "base" model doesn't answer questions

Ask GPT-2 a question directly. Real greedy output:

Prompt: `What is the capital of France?`

> *(newline)* The capital of France is Paris. *(newline)* The capital of France is Paris. *(newline)* The capital of France is Paris. *(newline)* The...

It continues the text the way documents online often do. It doesn't hold a conversation.

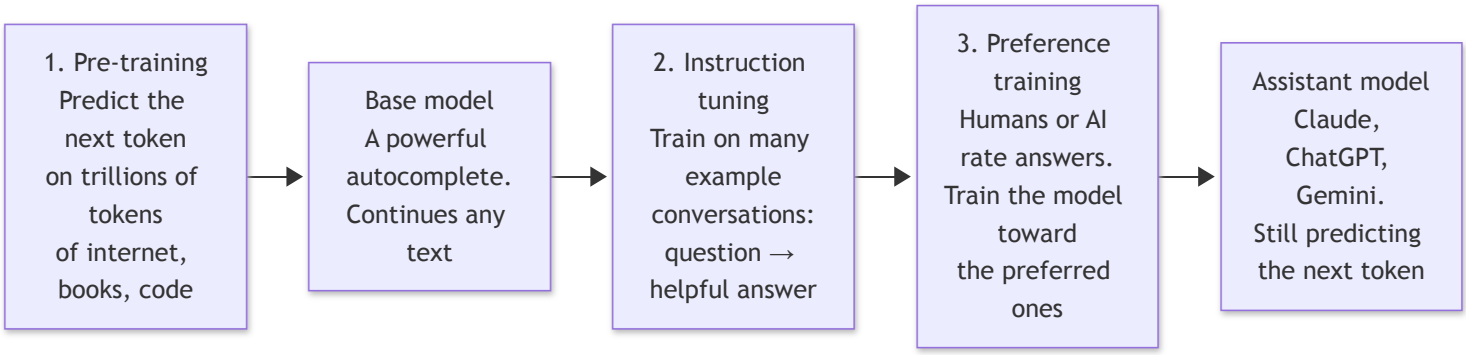
Now **change the format** so that an answer is the natural next text:

Prompt: `Q: What is the capital of France?\nA:`

> The capital of France is Paris.

The model didn't change. The prompt **made the answer the most likely continuation**. That's the whole idea behind prompting: you are setting up a document whose most likely next part is the thing you want.

How assistants are made



Steps 2 and 3 don't replace prediction. They change **what the model treats as likely**, so that a helpful, honest, well-formatted answer becomes the most likely continuation of a conversation.

A chat is one long document

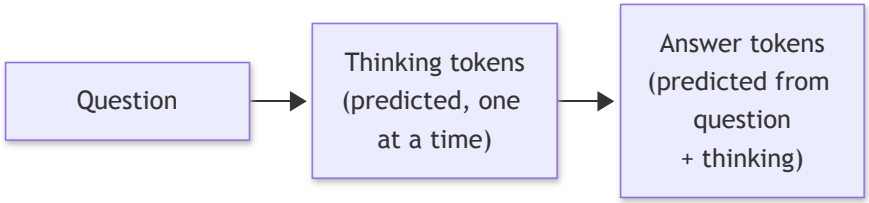
When you send a chat with roles, the provider joins it into one token sequence with special tokens marking the turns (Session 1). Conceptually it looks like this (illustrative, not any provider's exact format):

```
<|system|> You are a helpful coding assistant.
<|user|> What does `git rebase` do?
<|assistant|> ▌  ← the model predicts from here, one token at a time,
                   until it predicts an "end of turn" token
```

The model is completing the assistant's part of a transcript. A system prompt works because it sits at the top of the document and shapes every prediction after it.

Reasoning or "thinking" models

Models that "think before answering" still use the same loop. They are trained to **predict reasoning tokens first**, then the answer. Those reasoning tokens become context, so the final answer is predicted from a context that already contains the working-out.



That's why thinking improves hard answers, why it takes longer, and why thinking tokens are **billed as output** even when you don't see them.

7. One idea, many behaviours

This is the main point of the session. Each behaviour below follows from *"predict the next token, one step at a time"*.

Behaviour you've seen	Explanation from the one idea	Real example (GPT-2)
Fluent, confident writing	It was trained to produce text that looks like real text. Fluency is exactly what it optimises	Every output in this doc is grammatical, even the wrong ones
Hallucination (confident, made-up facts)	It predicts what text **like this usually** says, not what is true. If the prompt assumes something false, a plausible continuation still exists	`The 2019 Nobel Prize in Physics was awarded to the Nigerian scientist who discovered the first atomic bomb.` (No Nigerian won that prize, and no one person "discovered" the bomb)
Repeating common but outdated facts	Probability follows how often things appear in training	`France: Paris / Kenya: Nairobi / Ghana: Accra / Nigeria:` → `Lag` (start of *Lagos*) is in the top 2. Abuja has been the capital since 1991, but Lagos appears far more often in text

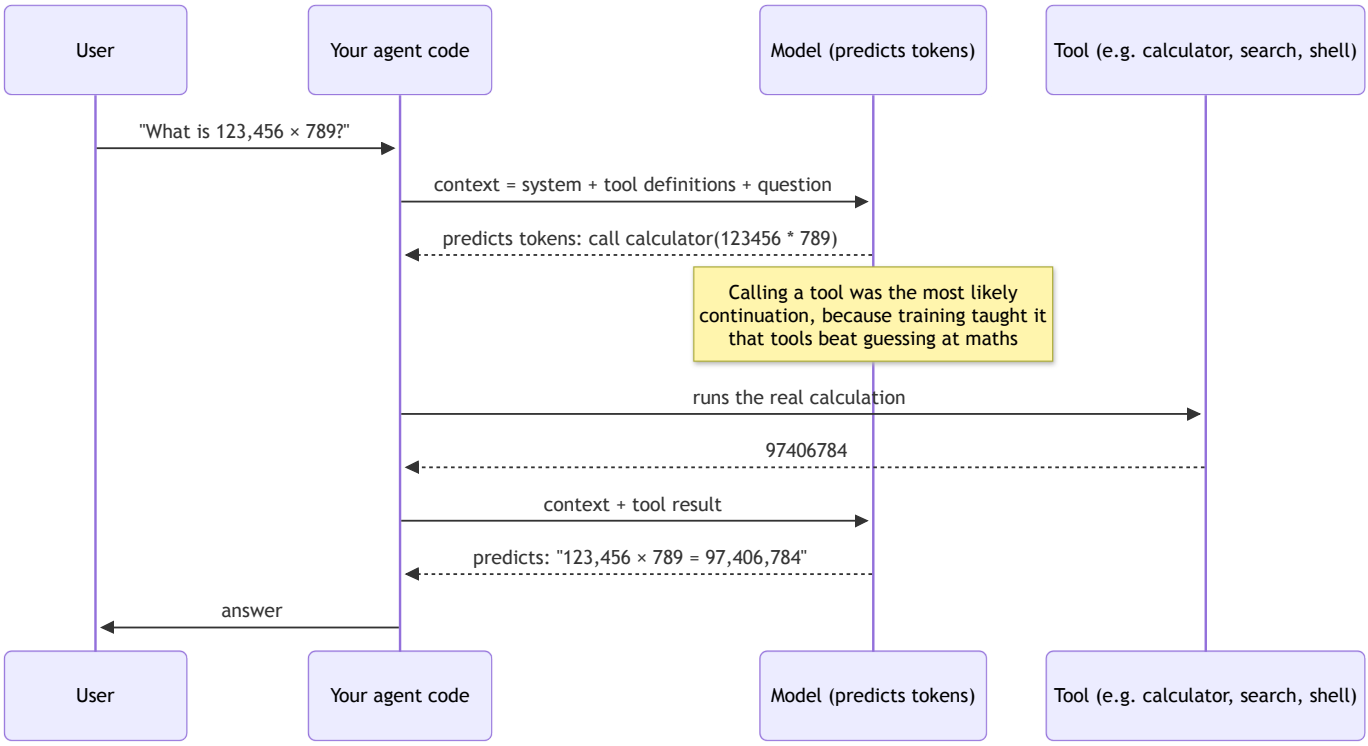
Behaviour you've seen	Explanation from the one idea	Real example (GPT-2)
	text, not what is currently correct	
Weak arithmetic	Numbers are tokens and patterns, not quantities. There's no calculator inside	`2 + 2 =` → `3` 10.2%, `1` 9.2%, `2` 8.4%, `0` 7.5%, `4` only **5.5%**
Different answer every time	Sampling picks from a distribution, so variation is built in	Three different "Once upon a time" stories in Section 5
Repetition loops	Repeating a phrase makes it more likely, which makes it repeat	`I'm not a bad person. I'm not a bad person...`
Prompt wording matters	Different wording means different context, which means different probabilities	`What is the capital...?` repeats itself. `Q: ... A:` answers
Examples are copied (few-shot)	The strongest signal in the context is a pattern to continue, sometimes including the example's answer	One positive example, then `'The food was cold and the waiter was rude.' Sentiment:` → `positive` **61%**, `negative` 17%. The small model copied the label from the example instead of judging the review
Mistakes snowball	It can't go back. Each wrong token becomes context for the next one	A wrong first step in a calculation leads to a confidently wrong result
"Think step by step" helps	Writing the working-out first adds useful context before the answer is predicted	The basis of reasoning models (Section 6)
Knowledge cutoff	It only knows patterns from its training text. Anything newer simply isn't there	Unless you put it in the context (search, documents, tools)
Good at format,	These are strong, repeated patterns in the training text	`Monday, Tuesday, Wednesday,` → `Thursday` 79%

Behaviour you've seen	Explanation from the one idea	Real example (GPT-2)
structure and style		
Tool use and agents	The model predicts tokens that **look like** a tool call. Your code runs the tool and adds the result to the context	See below

> **Important:** Modern assistant models make far fewer of these mistakes than GPT-2. They were trained much more, on better data, with feedback. But the **causes** are the same, so the same problems still appear at the edges: rare facts, tricky arithmetic, long chains of reasoning, false premises.

How an agent tool call actually works

The model can't run code or browse the web. It can only predict tokens. A tool call is **text the model predicts** in a format your program recognises.

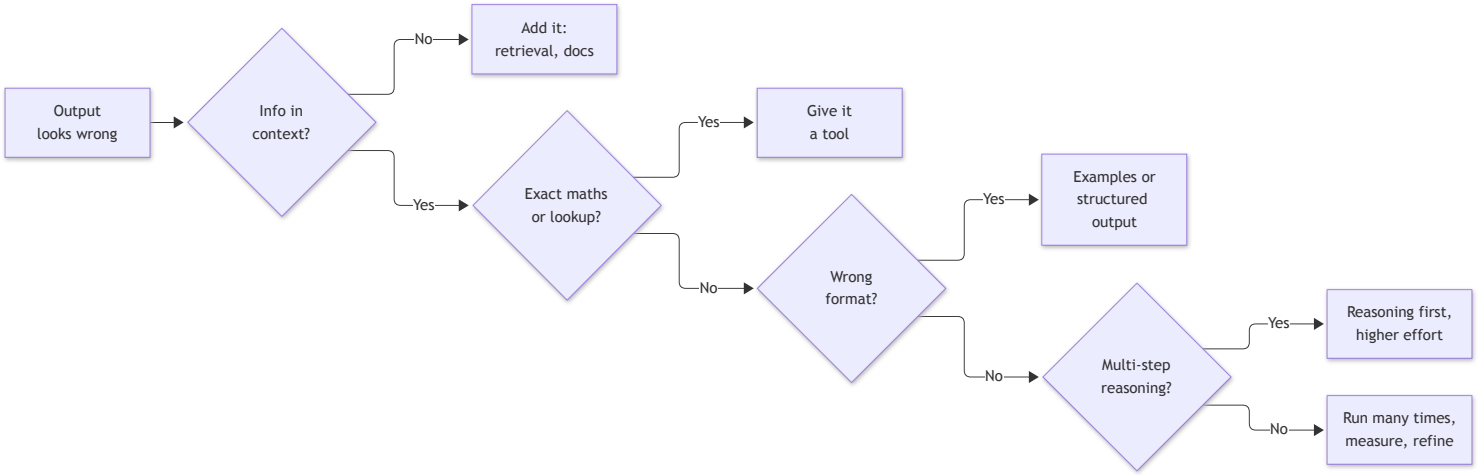


Everything we build with agents depends on this: **the model predicts, your code acts, and the results go back into the context** for the next prediction.

8. What this means for builders

Each practice below works **because** of how prediction works.

Practice	Why it works
Put the facts in the context (RAG, documents, search results)	The model predicts from the context. Correct facts in the context make correct continuations likely, and reduce hallucination
Give examples of the output you want	Examples are the strongest pattern to continue. Use varied examples, since it may copy them too closely (see the sentiment example)
Ask for reasoning before the answer	Reasoning tokens become context for the answer. Asking for the answer first means the reasoning can only justify it after the fact
Use tools for maths, dates, lookups and code execution	Prediction is unreliable for exact computation. Tools are exact
Use structured outputs or constrained decoding for JSON	Invalid tokens are blocked during generation, so the output always parses
Validate outputs in code	Every output is a sample from a distribution, so assume some will be wrong
Test with many runs, not one (evals)	One good answer doesn't prove the next one will be good. Measure how often it's right
Start fresh when a conversation goes off-track	Bad earlier tokens keep shaping new predictions. A clean context often works better than arguing
Keep the context clean and relevant	Irrelevant text in the context still influences predictions, and you pay for it (Session 1)
Ask the model to say when it doesn't know	Assistant training can make "I'm not sure" a likely answer, but only if the prompt makes it acceptable



9. Myths and reality

Myth	Reality
"It's just autocomplete, so it can't be smart."	It is next-token prediction, but predicting well across trillions of tokens forces the model to learn a lot about language, facts, code and reasoning. Simple objective, impressive result, real limits.

Myth	Reality
"It understands like a human."	It builds useful internal representations, but it has no goals, no senses, and no way to check its answers against the world unless we give it tools. Judge it by what it does reliably.
"It looks things up in a database."	A plain model has no database. Its knowledge is spread through its weights. Search and retrieval are **tools you add**.
"It remembers our past chats."	The model has no memory between requests. Any "memory" is the app putting earlier text back into the context.
"If it sounds confident, it's right."	Confidence in the writing is just likely-sounding text. A fluent sentence and a true sentence are produced the same way.
"Temperature 0 makes it fully deterministic."	It makes it much more consistent, but small numerical differences on servers can still change results. Don't rely on identical outputs.
"It plans the whole answer first."	It produces one token at a time. Any planning has to happen within that process, or in thinking tokens it writes first.

10. Hands-on demos

Demo 1: See the probabilities yourself (GPT-2 on a laptop)

```
pip install torch transformers
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")
model.eval()

def next_token_probs(prompt, k=8, temperature=1.0):
    ids = tok(prompt, return_tensors="pt").input_ids
    with torch.no_grad():
        logits = model(ids).logits[0, -1] / temperature
    probs = torch.softmax(logits, dim=-1)
    top_p, top_i = probs.topk(k)
    for p, i in zip(top_p, top_i):
        print(f"{tok.decode([i])!r:>14}  {p.item() * 100:6.2f}%")

next_token_probs("Monday, Tuesday, Wednesday,")
next_token_probs("I went to the bank to")
next_token_probs("We sat on the river bank and")
```

Try with the group: change `temperature` to `0.2` and `1.5` and watch the distribution sharpen and flatten.

Demo 2: Watch it write, one token at a time

```
ids = tok("Once upon a time", return_tensors="pt").input_ids

for step in range(8):
    with torch.no_grad():
        probs = torch.softmax(model(ids).logits[0, -1], dim=-1)
    top_p, top_i = probs.topk(3)
    options = ", ".join(f"{tok.decode([i])!r} {p.item() * 100:.1f}%" for p, i in zip(top_p, top_i))
    print(f"{tok.decode(ids[0])!r:<50} -> {options}")
    ids = torch.cat([ids, top_i[:1].view(1, 1)], dim=1)  # greedy: take the top token
```

Demo 3: Greedy vs sampling vs hot sampling

```
def generate(prompt, **kwargs):
    ids = tok(prompt, return_tensors="pt").input_ids
    out = model.generate(ids, max_new_tokens=25, pad_token_id=tok.eos_token_id, **kwargs)
    return tok.decode(out[0][ids.shape[1]:])

print("GREEDY:", generate("Once upon a time", do_sample=False))
for seed in [1, 2, 3]:
    torch.manual_seed(seed)
    print(f"SAMPLE {seed}:", generate("Once upon a time", do_sample=True, temperature=0.9, top_k=0))
torch.manual_seed(1)
print("HOT:", generate("Once upon a time", do_sample=True, temperature=2.0, top_k=0))
```

Demo 4: Same prompt, several runs, with Claude

Run the same request three times at two temperatures. This uses Claude Haiku 4.5, which still accepts `temperature`:

```
import anthropic

client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY

prompt = "Write a one-line slogan for a jollof rice restaurant in Lagos."

for temperature in [0.0, 1.0]:
    print(f"\n--- temperature {temperature} ---")
    for run in range(3):
        response = client.messages.create(
            model="claude-haiku-4-5",
            max_tokens=100,
            temperature=temperature,
            messages=[{"role": "user", "content": prompt}],
        )
        print(f"run {run + 1}:", response.content[0].text.strip())
```

Demo 5: No code needed (any chat app)

1. Ask *"Give me a random number between 1 and 10"* in five new chats. How random is it? Which numbers show up most?

2. Ask a question with a false premise, e.g. *"Why did Nigeria move its capital to Ibadan in 2005?"*. Does the model correct the premise or play along?
3. Regenerate the same creative prompt three times and compare.
4. Ask for a multiplication of two 6-digit numbers, once with *"answer only"* and once with *"work it out step by step"*. Check both with a calculator.

11. Takeaways

1. **An LLM predicts the next token**, using the whole context, as a probability over its entire vocabulary.
2. **Text is generated one token at a time.** Each chosen token becomes context for the next step, and nothing is revised once chosen.
3. **Training is a guessing game:** predict the next token in trillions of tokens of real text and adjust when wrong. Broad abilities come out of doing this well.
4. **Decoding turns probabilities into text.** Greedy is repetitive, sampling is varied, temperature controls how adventurous it is. That's why answers change between runs.
5. **Assistants are still predictors.** Instruction and preference training make helpful answers the likely continuation, and a chat is one long document the model completes.
6. **Most behaviours follow from this:** fluency, hallucination, arithmetic errors, copying examples, repetition, snowballing mistakes, the value of reasoning first, and tool calls.
7. **Build around it:** put facts in the context, give tools for exact work, use structured outputs, validate, and test over many runs.

Discussion questions

1. GPT-2 suggested *Lagos* as Nigeria's capital. What does that tell you about where a model's "facts" come from? How would you make a Nigerian civic-information bot reliable?
2. In the sentiment example, the model copied the example's label. How would you design few-shot examples to avoid that?
3. Why does asking for an answer *before* the explanation often give worse results than asking for the explanation first?
4. Your agent sometimes writes a perfect tool call and sometimes answers from memory instead. Using the prediction idea, what could cause the difference?
5. If a model is "just predicting", where does responsibility for correctness sit in an app you build?

Glossary

Term	Meaning
Next-token prediction	Estimating a probability for every possible next token, given the text so far
Probability distribution	The full list of probabilities over the vocabulary at one step (sums to 100%)
Logits	The raw scores the model outputs before they are turned into probabilities
Softmax	The function that converts logits into probabilities
Autoregressive	Generating one token at a time, with each output fed back as input
Context	All the tokens the model can see when making a prediction
Attention	The mechanism that lets each position use information from earlier tokens

Term	Meaning
Weights / parameters	The numbers learned in training that store the model's knowledge (124 million in GPT-2)
Pre-training	Learning to predict the next token on a huge general text collection
Loss	How wrong a prediction was. Training reduces it
Base model	A model after pre-training only. It continues text but doesn't follow instructions well
Instruction tuning	Training on example conversations so the model responds helpfully
Preference training (e.g. RLHF)	Training the model towards answers that people (or AI raters) prefer
Decoding / sampling	The method used to choose the next token from the probabilities
Greedy decoding	Always choosing the most likely token
Temperature	A setting that sharpens (low) or flattens (high) the probabilities before sampling
Top-k / top-p	Limit sampling to the k most likely tokens, or to the smallest set covering probability p
Constrained decoding	Blocking tokens that would break a required format, such as JSON
Hallucination	Fluent output that is not true or not supported by the context
Few-shot prompting	Putting examples of the task in the prompt so the model continues the pattern
Knowledge cutoff	The date after which the model's training text has no information

Further reading and watching

- **3Blue1Brown**, *But what is a GPT? Visual intro to transformers* (YouTube, 2024). The best visual explanation of a single prediction step.
- **Stephen Wolfram**, *What Is ChatGPT Doing … and Why Does It Work?* (2023). A long, readable essay built around exactly this session's one idea.
- Vaswani et al. (2017), *Attention Is All You Need*. The paper that introduced the transformer.
- Ouyang et al. (2022), *Training language models to follow instructions with human feedback* (InstructGPT). How a base model becomes an assistant.